

CO5-AT-2

MLA0201-Fundamentals Of Machine Learning

Pseudocode Writing Task (Algorithm Design)

QUESTION 1: Monte Carlo Sampling for Probability Estimation

Question:

A logistics company wants to estimate the probability of on-time deliveries under uncertain traffic conditions using Monte Carlo Sampling.

Aim:

To estimate the probability of an **on-time delivery** by generating random delivery conditions and performing many delivery simulations.

Pseudocode:

ALGORITHM Monte_Carlo_Delivery

START

1. Set the number of simulations N.
 2. Set OnTimeCount = 0.
 3. FOR each simulation from 1 to N:
 - Generate a random traffic condition.
 - Generate a random delivery time.
 - IF delivery time <= allowed delivery time THEN
 - OnTimeCount = OnTimeCount + 1.
 - END IF
 - END FOR
 4. Calculate Probability = OnTimeCount / N.
 5. Display the estimated probability.
- END

Important Point:

The number of simulations should be large enough to obtain a reliable probability estimate.

Why Is It Important?

Monte Carlo Sampling uses random experiments. A small number of simulations may produce an inaccurate result. Increasing the number of trials generally makes the estimated probability more stable.

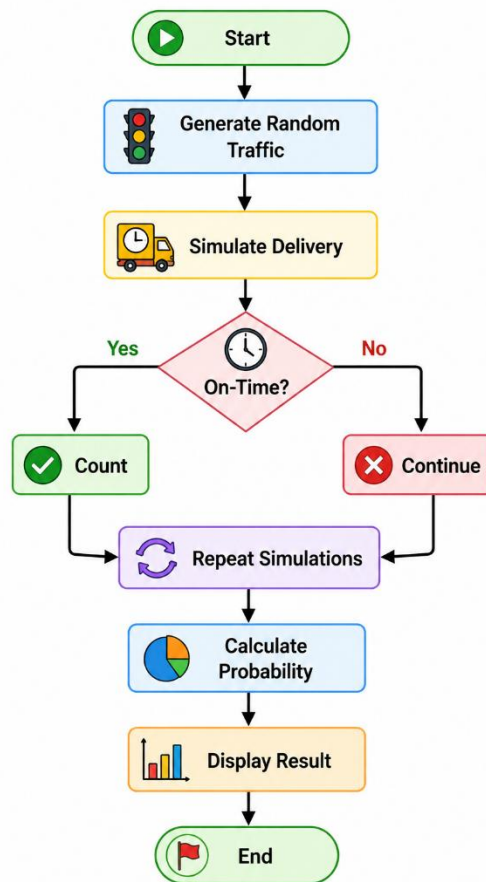
Sample Input:

Number of Simulations = 10000

Maximum Delivery Time = 60 minutes

Traffic Conditions = Light, Medium, Heavy.

Simple Flow:



Code :

```
import random

N = 10000
MAX_TIME = 60

on_time_count = 0

for i in range(N):
    traffic = random.choice(["Light", "Medium", "Heavy"])

    if traffic == "Light":
        delivery_time = random.uniform(20, 40)
    elif traffic == "Medium":
        delivery_time = random.uniform(30, 55)
    else:
        delivery_time = random.uniform(45, 80)

    if delivery_time <= MAX_TIME:
        on_time_count += 1

probability = on_time_count / N

print("Monte Carlo Delivery Simulation")
print("Number of Simulations:", N)
print("On-Time Deliveries:", on_time_count)
print("Estimated Probability:", round(probability, 4))
print("Estimated Probability (%):", round(probability * 100, 2), "%")
```

Sample Output:

```
Monte Carlo Delivery Simulation
Number of Simulations: 10000
On-Time Deliveries: 8083
Estimated Probability: 0.8083
Estimated Probability (%): 80.83 %
```

Conclusion:

Monte Carlo Sampling estimates the probability of on-time delivery by repeating random delivery experiments. It is useful when traffic and delivery conditions are uncertain.

QUESTION 2: ϵ -Greedy Algorithm for K-Armed Bandit

Question:

An online streaming platform recommends one of several movies to maximize user engagement. The platform must balance exploration and exploitation.

Aim:

To select the best movie recommendation by balancing **exploration** of new movies and **exploitation** of movies with high estimated rewards.

Pseudocode:

```
ALGORITHM Epsilon_Greedy
START
1. Set the number of movies K.
2. Initialize reward estimate of every movie to 0.
3. Initialize selection count of every movie to 0.
4. Set epsilon value.
5. FOR each user interaction:
    Generate a random number.
    IF random number < epsilon THEN
        Select a random movie.
        // Exploration
    ELSE
        Select the movie with highest reward estimate.
        // Exploitation
    END IF
6. Observe the user reward.
7. Update the selected movie's reward estimate.
8. Repeat for all interactions.
9. Select the movie with the highest estimated reward.
10. Display the best movie.
END
```

Important Point:

The value of ϵ controls the balance between exploration and exploitation.

Why Is It Important?

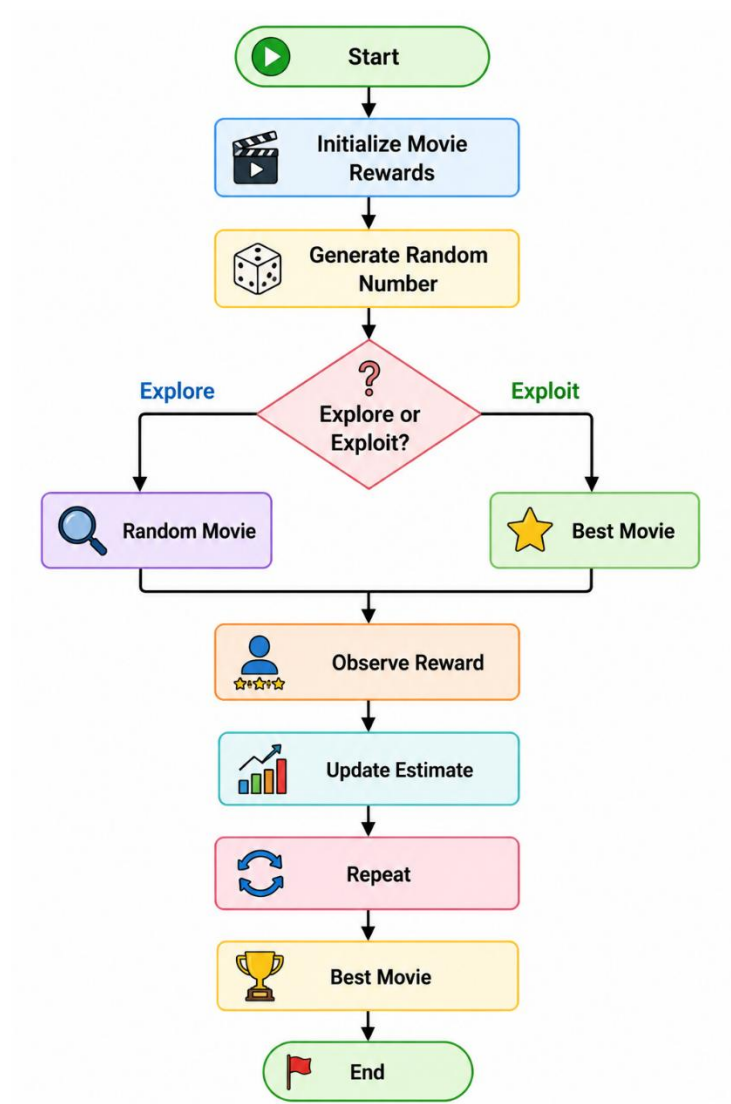
If ϵ is too high, the system explores too much and may not exploit the best movie. If ϵ is too low, the system may never discover a better movie.

For example:

$$\epsilon = 0.10$$

10% $\rightarrow$ Exploration

90% $\rightarrow$ Exploitation

Simple Flow:**Sample Input:**

Number of Movies = 5

Epsilon = 0.10

User Interactions = 10000

Code :

```
import random
K = 5
epsilon = 0.10
interactions = 10000
true_rewards = [0.30, 0.50, 0.70, 0.40, 0.60]
reward_estimates = [0.0] * K
counts = [0] * K
for i in range(interactions):
    if random.random() < epsilon:
        movie = random.randint(0, K - 1)
    else:
        movie = reward_estimates.index(max(reward_estimates))
    reward = 1 if random.random() < true_rewards[movie] else 0
    counts[movie] += 1

    reward_estimates[movie] += (
        reward - reward_estimates[movie]
    ) / counts[movie]
best_movie = reward_estimates.index(max(reward_estimates))
print("Epsilon-Greedy Movie Recommendation")
print("Epsilon:", epsilon)
print("Total Interactions:", interactions)
print()
for i in range(K):
    print(
        "Movie", i + 1,
        "| Selections:", counts[i],
        "| Estimated Reward:",
        round(reward_estimates[i], 4)
    )
print()
print("Best Movie: Movie", best_movie + 1)
print(
    "Highest Expected Reward:",
    round(reward_estimates[best_movie], 4)
)
```

Sample Output:

```
*** Epsilon-Greedy Movie Recommendation
Epsilon: 0.1
Total Interactions: 10000

Movie 1 | Selections: 212 | Estimated Reward: 0.2972
Movie 2 | Selections: 205 | Estimated Reward: 0.4146
Movie 3 | Selections: 9057 | Estimated Reward: 0.699
Movie 4 | Selections: 213 | Estimated Reward: 0.3944
Movie 5 | Selections: 313 | Estimated Reward: 0.5911

Best Movie: Movie 3
Highest Expected Reward: 0.699
```

Conclusion:

The ϵ -greedy algorithm learns which movie produces the highest engagement while still exploring other recommendations.

QUESTION 3: Value Iteration for Robot Path Planning**Question:**

A warehouse robot must determine the optimal path to reach a destination while maximizing rewards and avoiding obstacles.

Aim:

To find the **optimal value of each state** and the **best action** for a warehouse robot using Value Iteration.

Pseudocode:

ALGORITHM Value_Iteration

START

1. Define all warehouse states.
 2. Define actions: UP, DOWN, LEFT, RIGHT.
 3. Define the obstacle state.
 4. Define the destination state.
 5. Initialize value $V(s) = 0$ for all states.
 6. Set discount factor Gamma.
 7. REPEAT:
 - Set Delta = 0.
 - FOR each non-terminal state:
 - Calculate the value of every possible action.
 - Select the maximum action value.
 - Update the state value.
 - Calculate the change in value.
 - END FOR
 - UNTIL Delta is smaller than the threshold.
 8. For every state:
 - Select the action with the highest value.
 9. Display the optimal value function and policy.
- END

Important Point:

The Bellman optimality equation is the key part of Value Iteration.

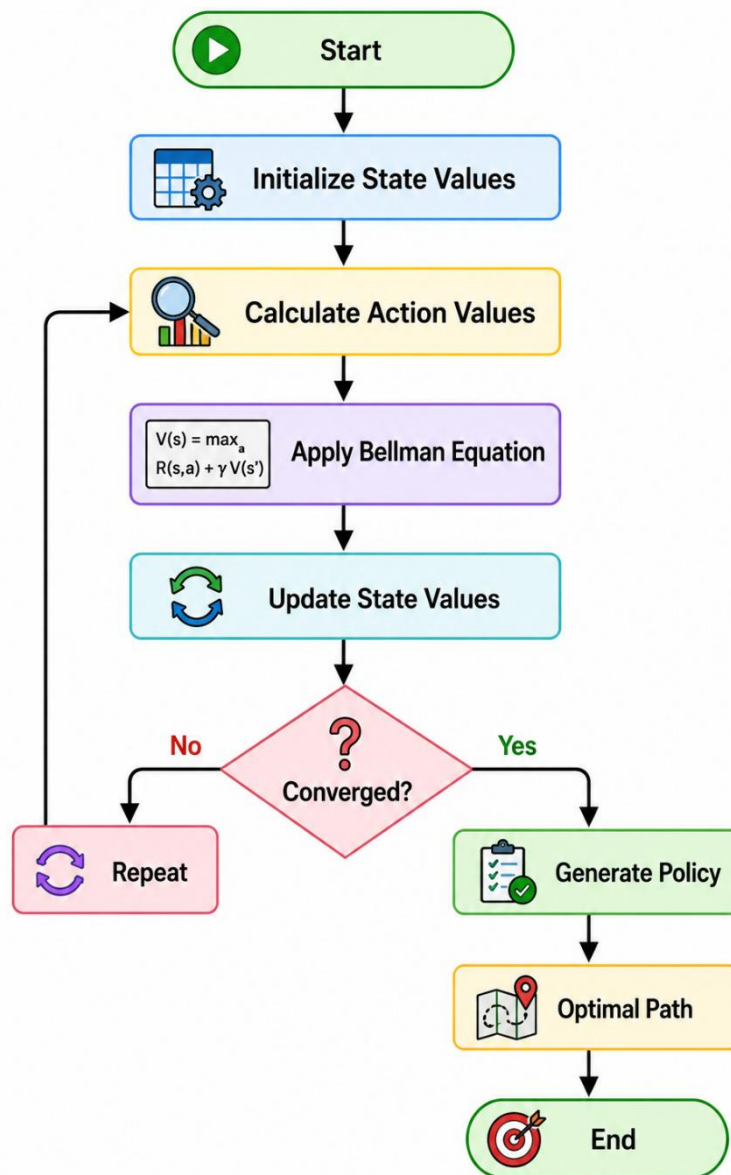
Why Is It Important?

The Bellman equation allows the robot to determine which action gives the highest future reward. Repeating this update eventually produces the optimal values and policy.

Bellman Equation:

$$V(s) = \max_a [R(s, a) + \gamma V(s')]$$

Simple Flow:



Sample Input:

Grid Size = 4×4

Start = (0,0)

Goal = (3,3)

Obstacle = (1,1)

Actions = UP, DOWN, LEFT, RIGHT

Discount Factor = 0.9

Code:



```
import numpy as np
rows, cols = 4, 4
goal = (3, 3)
obstacle = (1, 1)
actions = {
    "UP": (-1, 0),
    "DOWN": (1, 0),
    "LEFT": (0, -1),
    "RIGHT": (0, 1)
}
gamma = 0.9
threshold = 0.001
V = np.zeros((rows, cols))
def next_state(state, action):
    r, c = state
    dr, dc = actions[action]
    nr = r + dr
    nc = c + dc
    if nr < 0 or nr >= rows or nc < 0 or nc >= cols:
        return state
    if (nr, nc) == obstacle:
        return state
    return (nr, nc)
def reward(state):
    if state == goal:
        return 10
    return -1
while True:
    delta = 0
    new_V = V.copy()
    for r in range(rows):
        for c in range(cols):
            state = (r, c)
            if state == goal or state == obstacle:
                continue
            values = []
            for action in actions:
                ns = next_state(state, action)
                value = reward(ns) + gamma * V[ns]
                values.append(value)
            new_V[state] = max(values)
            delta = max(
                delta,
                abs(new_V[state] - V[state])
            )

    V = new_V
    if delta < threshold:
        break
policy = {}
for r in range(rows):
    for c in range(cols):
        state = (r, c)
        if state == goal:
            policy[state] = "GOAL"
            continue
        if state == obstacle:
            policy[state] = "OBSTACLE"
            continue
        values = {}
        for action in actions:
            ns = next_state(state, action)
            values[action] = reward(ns) + gamma * V[ns]
        policy[state] = max(values, key=values.get)
print("Optimal Value Function:")
print(np.round(V, 2))
print("\nOptimal Policy:")
for r in range(rows):
    print([policy[(r, c)] for c in range(cols)])
```


Sample Output:

```
*** Optimal Value Function:
[[ 1.81  3.12  4.58  6.2 ]
 [ 3.12  0.    6.2   8.  ]
 [ 4.58  6.2   8.   10. ]
 [ 6.2   8.   10.   0.  ]]

Optimal Policy:
['DOWN', 'RIGHT', 'DOWN', 'DOWN']
['DOWN', 'OBSTACLE', 'DOWN', 'DOWN']
['DOWN', 'DOWN', 'DOWN', 'DOWN']
['RIGHT', 'RIGHT', 'RIGHT', 'GOAL']
```

Conclusion:

Value Iteration repeatedly updates state values until they converge. The resulting policy gives the robot the best action for reaching the destination while avoiding obstacles.

QUESTION 4: Temporal Difference (TD) Learning for Game Score Prediction

Question:

A game AI agent learns the value of each game state based on rewards received after every move.

Aim:

To estimate the value of game states by updating the value after every move using **Temporal Difference Learning**.

Pseudocode:

ALGORITHM TD_Learning

START

1. Initialize $V(s) = 0$ for all states.

2. Set learning rate Alpha.

3. Set discount factor Gamma.

4. FOR each game episode:

 Observe the current state S.

 REPEAT:

 Take an action.

 Observe reward R.

 Observe next state S'.

 Update $V(S)$ using TD rule.

 Set $S = S'$.

 UNTIL the game ends.

END FOR

5. Display the learned state values.

END

Important Point:

The TD learning rule updates the current state immediately using the next state's estimated value.

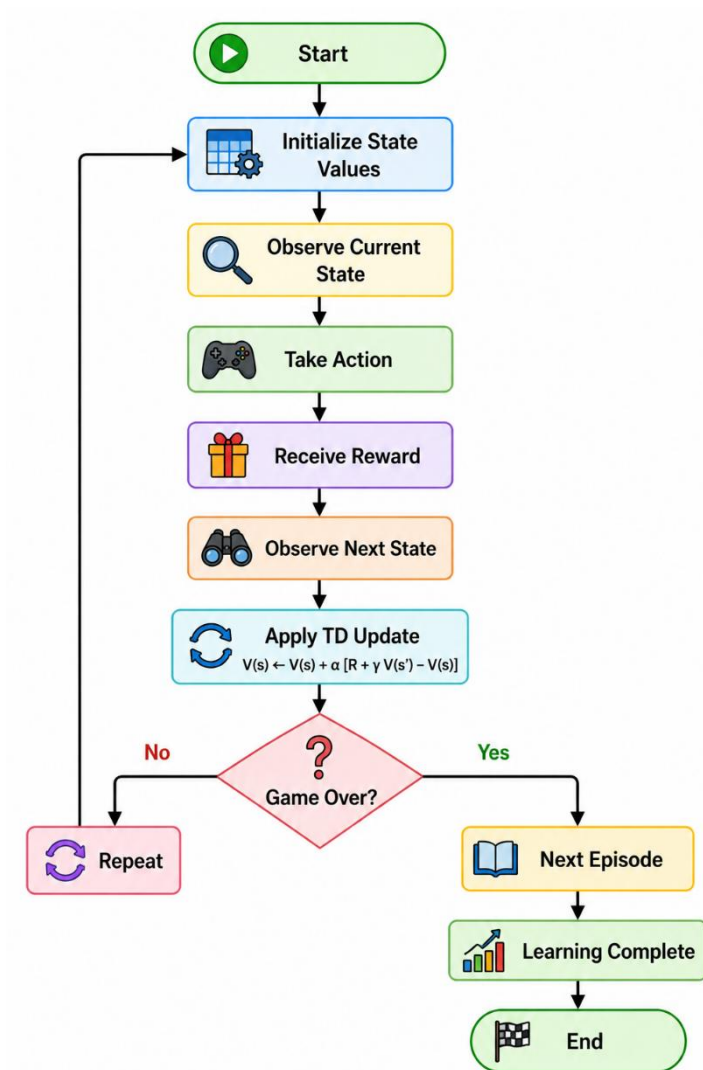
Why Is It Important?

The AI does not need to wait until the entire game ends. It can learn after every move, making TD Learning faster and suitable for sequential decision-making.

TD Learning Equation:

$$V(S) \leftarrow V(S) + \alpha[R + \gamma V(S') - V(S)]$$

Simple Flow:



Sample Input:

States = Start, Middle, NearGoal, Goal

Learning Rate = 0.1

Discount Factor = 0.9

Episodes = 1000

Code:

```
import random
alpha = 0.1
gamma = 0.9
episodes = 1000
states = ["Start", "Middle", "NearGoal", "Goal"]
V = {state: 0.0 for state in states}
for episode in range(episodes):
    state = "Start"
    while state != "Goal":
        if state == "Start":
            next_state = "Middle"
            reward = 0
        elif state == "Middle":
            next_state = "NearGoal"
            reward = 0
        elif state == "NearGoal":
            next_state = "Goal"
            reward = 10
        V[state] = V[state] + alpha * (
            reward +
            gamma * V[next_state] -
            V[state]
        )
        state = next_state
V["Goal"] = 0
print("TD Learning - Game State Values")
print()
for state in states:
    print(state, ":", round(V[state], 4))
print()
print("Learning completed after", episodes, "episodes.")
```

Sample Output:

```
... TD Learning - Game State Values

Start : 8.1
Middle : 9.0
NearGoal : 10.0
Goal : 0

Learning completed after 1000 episodes.
```

Conclusion:

TD Learning continuously improves state-value estimates after every move. It is useful for game AI because the agent can learn directly from experience without waiting for the complete episode.

QUESTION 5: Policy Iteration for Autonomous Drone Navigation

Question:

An autonomous drone must learn the optimal navigation policy to deliver packages while minimizing energy consumption.

Aim:

To find the optimal drone navigation policy by repeatedly performing **policy evaluation** and **policy improvement**.

Pseudocode:

ALGORITHM Policy_Iteration

START

1. Define all states.
 2. Define possible actions.
 3. Initialize a random policy.
 4. Initialize state values to zero.
 5. REPEAT
 - POLICY EVALUATION:
 - a. Calculate the value of each state according to the current policy.
 - b. Repeat until the values converge.
 - POLICY IMPROVEMENT:
 - c. For each state, calculate the value of every possible action.
 - d. Select the action with the highest value.
 - e. Update the policy.
 6. UNTIL the policy becomes stable.
 7. Display the optimal policy.
- END

Important Point:

Policy Iteration has two main stages: Policy Evaluation and Policy Improvement.

Why Is It Important?

Policy Evaluation determines how good the current policy is, while Policy Improvement makes the policy better. Repeating both steps eventually produces a stable optimal policy.

Sample Input:

Grid Size = 4×4

Start = (0,0)

Goal = (3,3)

Obstacle = (1,1)

Actions = UP, DOWN, LEFT, RIGHT

Discount Factor = 0.9

Code:

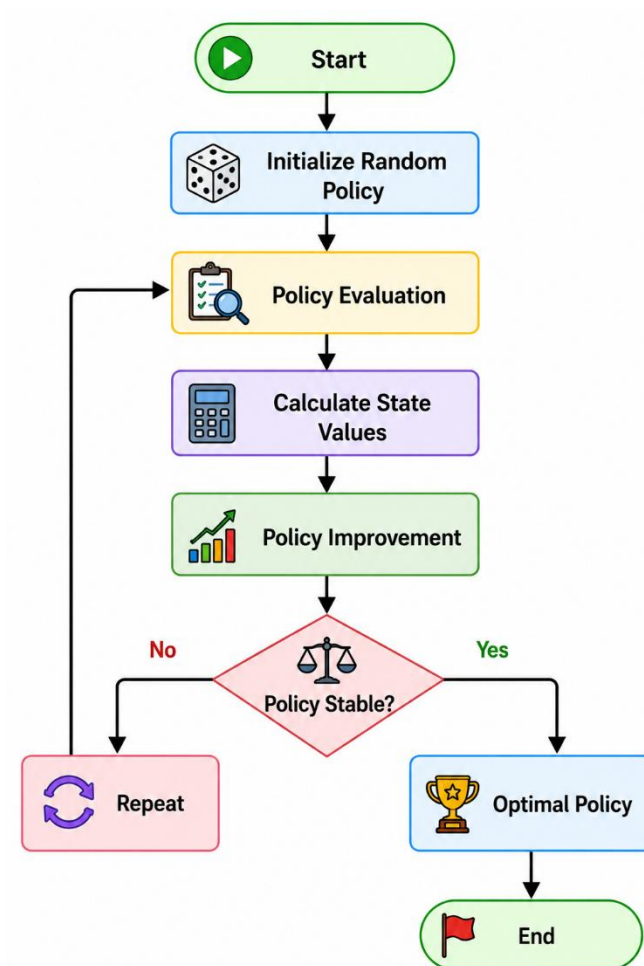
```
import numpy as np
rows, cols = 4, 4
goal = (3, 3)
obstacle = (1, 1)
actions = {
    "UP": (-1, 0),
    "DOWN": (1, 0),
    "LEFT": (0, -1),
    "RIGHT": (0, 1)
}
gamma = 0.9
theta = 0.001
V = np.zeros((rows, cols))
policy = {}
for r in range(rows):
    for c in range(cols):
        state = (r, c)
        if state == goal:
            policy[state] = "GOAL"
        elif state == obstacle:
            policy[state] = "OBSTACLE"
        else:
            policy[state] = "RIGHT"
def next_state(state, action):
    r, c = state
    dr, dc = actions[action]
    nr = r + dr
    nc = c + dc
    if nr < 0 or nr >= rows or nc < 0 or nc >= cols:
        return state
    if (nr, nc) == obstacle:
        return state
    return (nr, nc)
def reward(state):
    if state == goal:
        return 10
    return -1
stable = False
while not stable:
    # Policy Evaluation
    while True:
        delta = 0
        new_V = V.copy()
        for r in range(rows):
            for c in range(cols):
                state = (r, c)
                if state == goal or state == obstacle:
                    continue
                action = policy[state]
                ns = next_state(state, action)
                new_V[state] = reward(ns) + gamma * V[ns]
                delta = max(
                    delta,
                    abs(new_V[state] - V[state])
                )
        V = new_V
        if delta < theta:
            break
    # Policy Improvement
    stable = True
    for r in range(rows):
        for c in range(cols):
            state = (r, c)
            if state == goal or state == obstacle:
                continue
            old_action = policy[state]
            action_values = {}
            for action in actions:
                ns = next_state(state, action)
                action_values[action] = (
                    reward(ns) + gamma * V[ns]
                )
            best_action = max(
                action_values,
                key=action_values.get
            )
            policy[state] = best_action
            if old_action != best_action:
                stable = False
print("Optimal State Values:")
print(np.round(V, 2))
print("\nOptimal Drone Navigation Policy:")
for r in range(rows):
    print([policy[(r, c)] for c in range(cols)])
```

Sample Output:

```
*** Optimal State Values:
[[ 1.81  3.12  4.58  6.2 ]
 [ 3.12  0.    6.2  8.   ]
 [ 4.58  6.2   8.   10.  ]
 [ 6.2   8.   10.   0.   ]]

Optimal Drone Navigation Policy:
['DOWN', 'RIGHT', 'DOWN', 'DOWN']
['DOWN', 'OBSTACLE', 'DOWN', 'DOWN']
['DOWN', 'DOWN', 'DOWN', 'DOWN']
['RIGHT', 'RIGHT', 'RIGHT', 'GOAL']
```

Simple Flow:



Conclusion:

Policy Iteration finds an optimal navigation strategy by evaluating and improving the policy repeatedly. The final stable policy helps the drone reach its destination while considering movement cost and rewards.